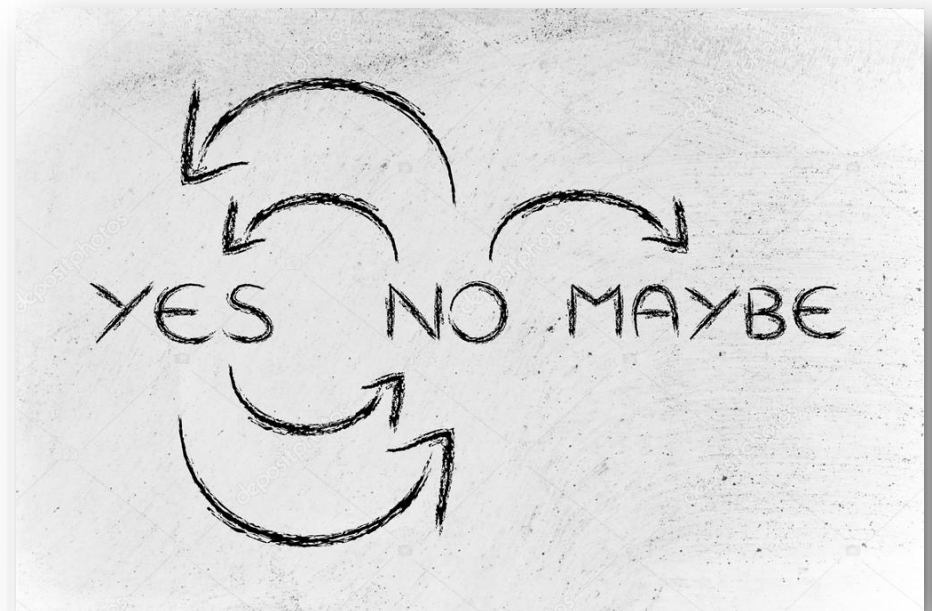


Künstliche Intelligenz / Maschinelles Lernen



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Trainieren von tiefen, probabilistischen Netzwerken



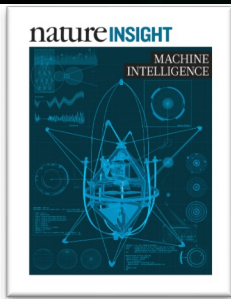
Basierende auf Folien von Pedro Domingos. Danke fürs Offenlegen ihrer Folien



TECHNISCHE
UNIVERSITÄT
DARMSTADT

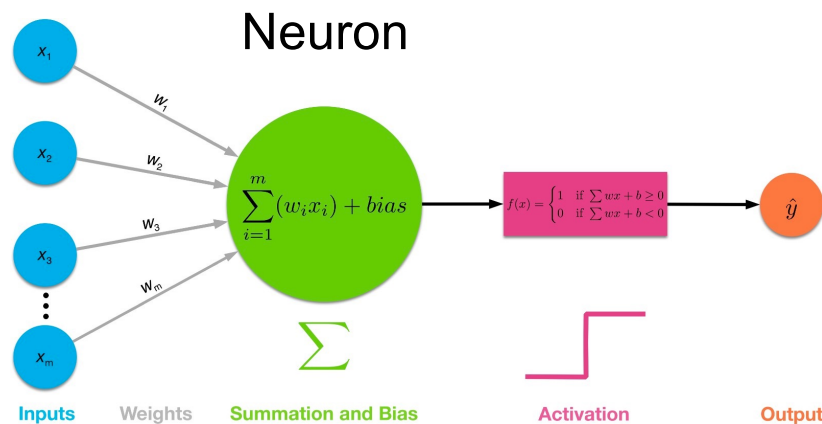


Deep Neural Networks

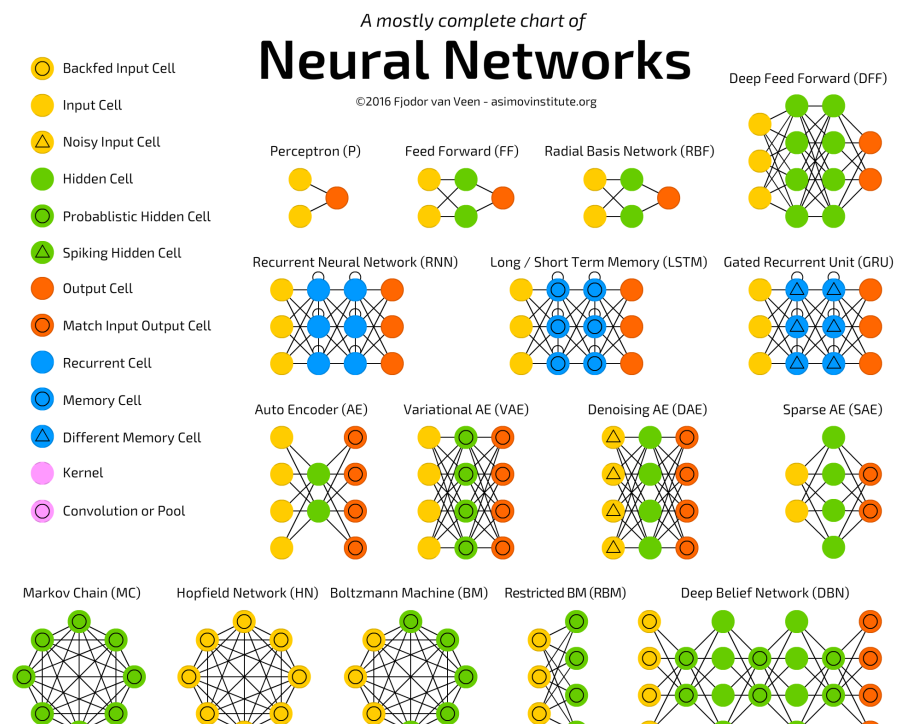


Potentiell mächtigere Darstellungen als “flache” Ansätze des Maschinellen Lernens

[LeCun, Bengio, Hinton Nature 521, 436–444, 2015]



Differenzierbare Programmierung



Tiefe Neuronale Netzwerke müssen keine Verbundwahrscheinlichkeit darstellen

MNIST



Train & Evaluate

SVHN

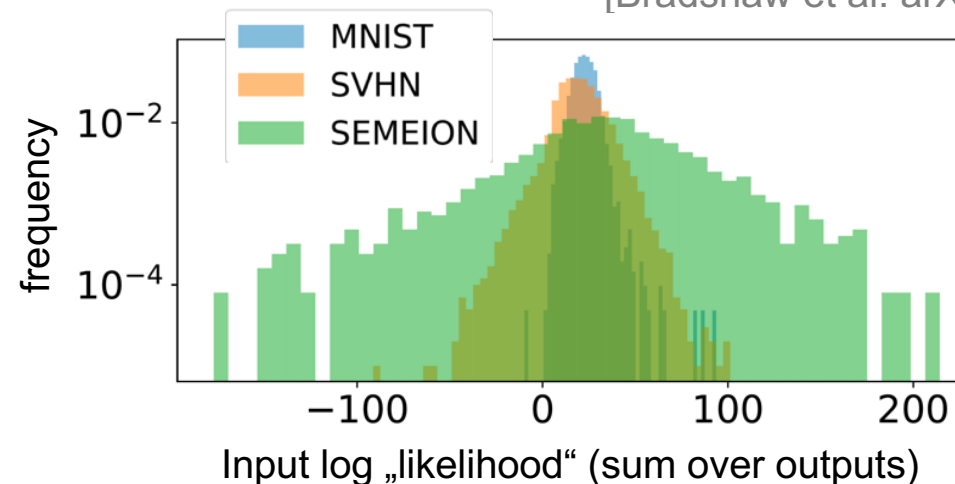


SEMEION



Transfer Testing

[Bradshaw et al. arXiv:1707.02476 2017]





**Tiefe Modelle, die wissen, was
sie nicht wissen!**

Können wir probabilistische Graphische Modelle mit tiefen Neuronalen Netzwerken verheiraten?

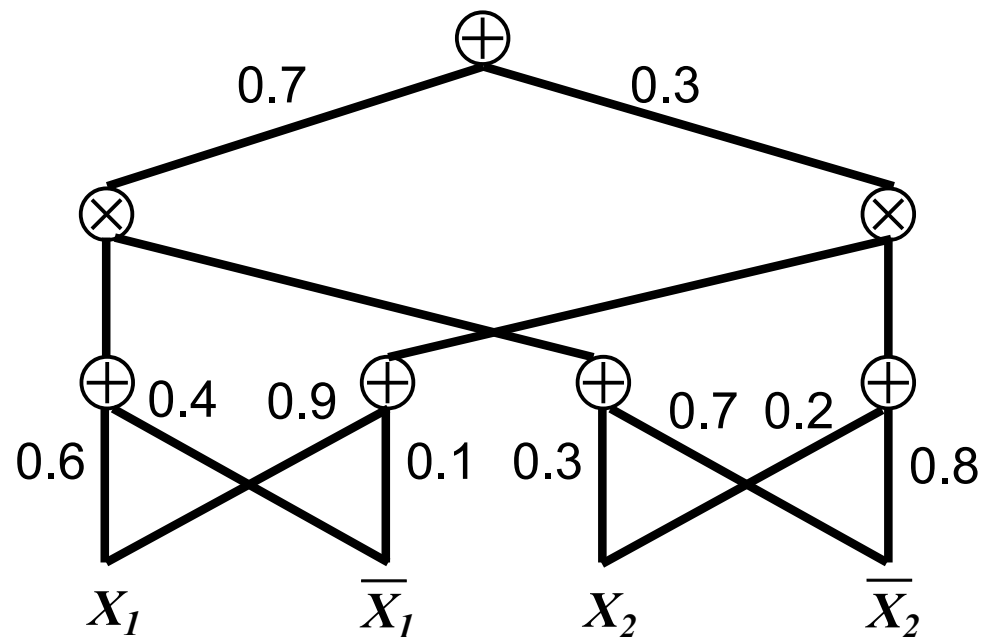
Judea Pearl, UCLA
Turing-Preis 2012



Tiefe Probabilistische Modellierung mittels Summen-Produkt Netzwerken

Adnan Darwiche
UCLA

Pedro Domingos
UW



Berechnungsgraphen
(ähnlich wie tiefe
neuronale Netzwerke)
für Verbund-
wahrscheinlichkeiten

Schlussfolgern ist linear in der Größe des Netzwerkes

Alternative Repräsentation: Bayes'sche Netzwerke als Tiefe Neuronale Netzwerke



TECHNISCHE
UNIVERSITÄT
DARMSTADT

X_1	X_2	$P(X)$
1	1	0.4
1	0	0.2
0	1	0.1
0	0	0.3

$$\begin{aligned} P(X) = & 0.4 \cdot I[X_1=1] \cdot I[X_2=1] \\ & + 0.2 \cdot I[X_1=1] \cdot I[X_2=0] \\ & + 0.1 \cdot I[X_1=0] \cdot I[X_2=1] \\ & + 0.3 \cdot I[X_1=0] \cdot I[X_2=0] \end{aligned}$$



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Alternative Repräsentation: Bayes'sche Netzwerke als Tiefe Neuronale Netzwerke



TECHNISCHE
UNIVERSITÄT
DARMSTADT

X_1	X_2	$P(X)$
1	1	0.4
1	0	0.2
0	1	0.1
0	0	0.3

$$\begin{aligned} P(X) = & \mathbf{0.4 \cdot I[X_1=1] \cdot I[X_2=1]} \\ & + 0.2 \cdot I[X_1=1] \cdot I[X_2=0] \\ & + 0.1 \cdot I[X_1=0] \cdot I[X_2=1] \\ & + 0.3 \cdot I[X_1=0] \cdot I[X_2=0] \end{aligned}$$



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Kurzform für die Indikatorfunktionen

X_1	X_2	$P(X)$
1	1	0.4
1	0	0.2
0	1	0.1
0	0	0.3

$$\begin{aligned}P(X) = & 0.4 \cdot X_1 \cdot X_2 \\ & + 0.2 \cdot X_1 \cdot \bar{X}_2 \\ & + 0.1 \cdot \bar{X}_1 \cdot X_2 \\ & + 0.3 \cdot \bar{X}_1 \cdot \bar{X}_2\end{aligned}$$



Marginalisierung

X_1	X_2	$P(X)$
1	1	0.4
1	0	0.2
0	1	0.1
0	0	0.3

$$q: X_1 = 1$$

$$\begin{aligned} P(q) &= 0.4 \cdot X_1 \cdot X_2 \\ &\quad + 0.2 \cdot X_1 \cdot \bar{X}_2 \\ &\quad + 0.1 \cdot \bar{X}_1 \cdot X_2 \\ &\quad + 0.3 \cdot \bar{X}_1 \cdot \bar{X}_2 \end{aligned}$$

$$\text{Set } X_1 = 1, \bar{X}_1 = 0, X_2 = 1, \bar{X}_2 = 1$$

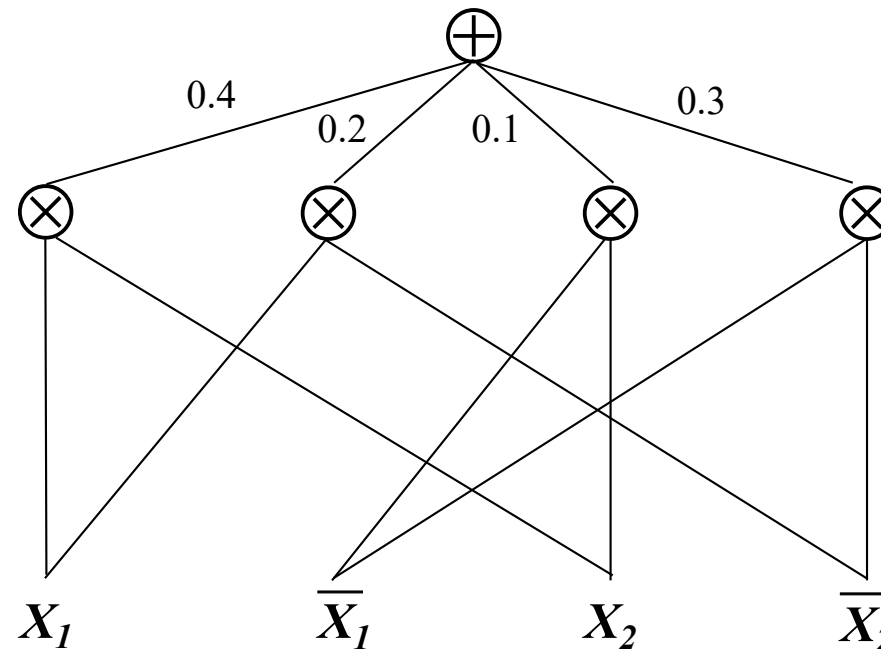
Marginalisierung von X_2 ist einfach, setze beide Werte von X_2 auf 1 ("X2 kann in beiden Zuständen sein")

Idee: Berechnungsgraphen, die Wahrscheinlichkeiten berechnen



TECHNISCHE
UNIVERSITÄT
DARMSTADT

X_1	X_2	$P(X)$
1	1	0.4
1	0	0.2
0	1	0.1
0	0	0.3



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Sum-Product Networks* (SPNs)

[Poon, Domingos UAI 2011]

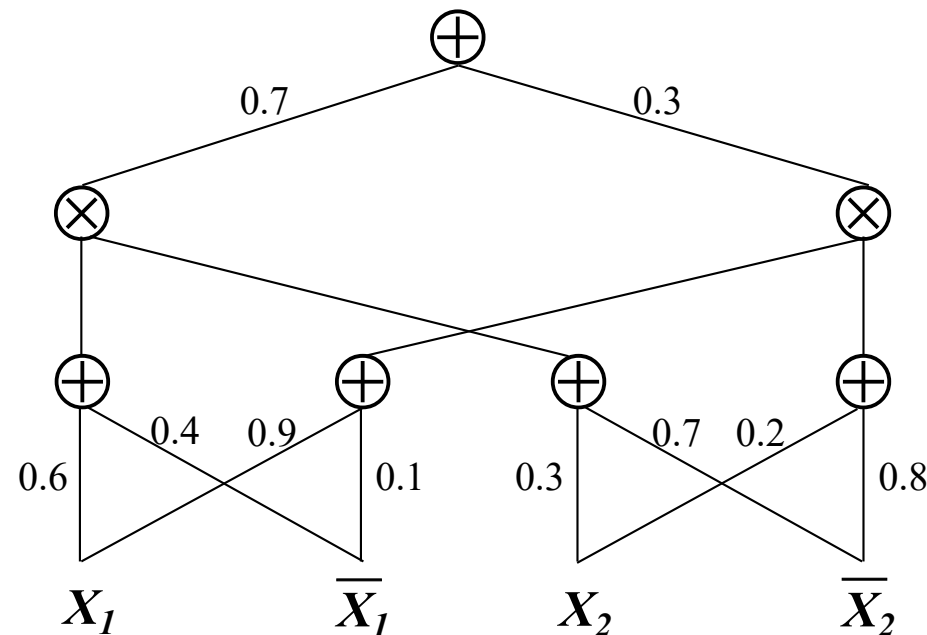


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Ein SPN **S** ist ein DAG mit Wurzel:

Knoten: Summe, Produkt, Eingabe

Gewichte an den Kanten sind Mischverteilung über den Kindern



* SPNs sind eine Instanz arithmetischer Schaltkreise (ACs). ACs wurden vor mehr als 20 Jahren in der KI-Literatur als handhabbare Darstellung von Wahrscheinlichkeitsverteilungen eingeführt. [Darwiche CACM 48(4):608-647 2001]

Schlussfolgern: **Linear in der Eingabegröße**

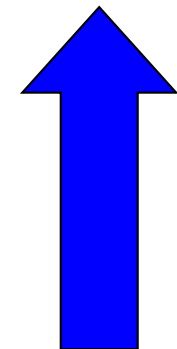
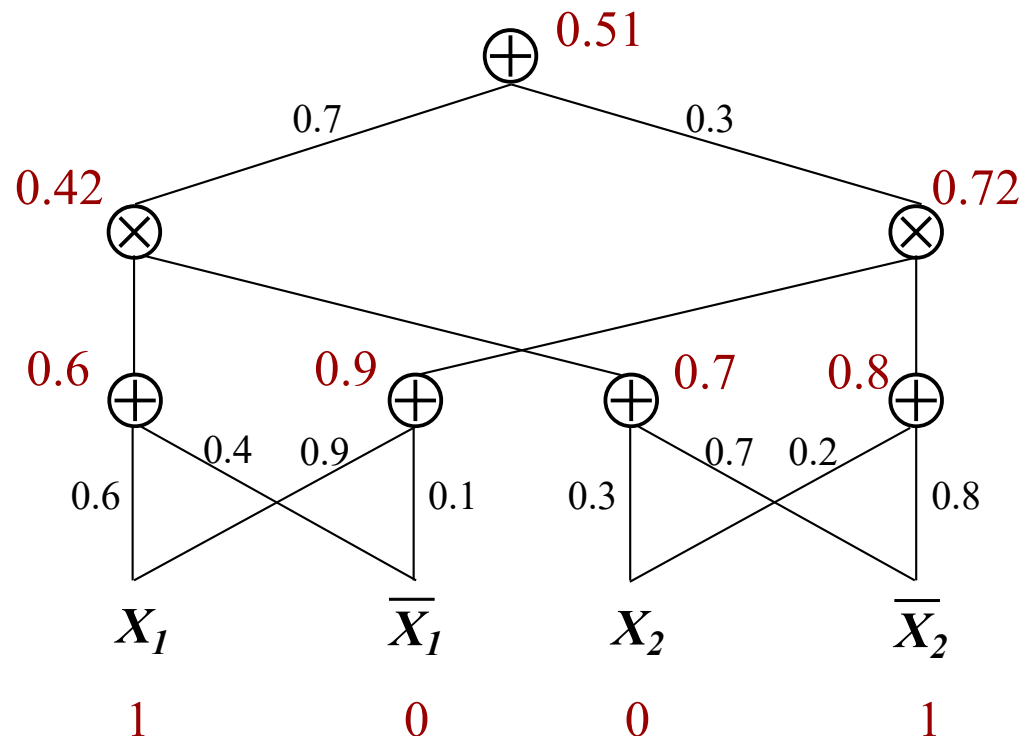


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Verbundwahrscheinlichkeit: $P(q)$

$q: X_1 = 1, X_2 = 0$

X_1	1
$\overline{X_1}$	0
X_2	0
$\overline{X_2}$	1



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Schlussfolgern: **Linear in der Eingabegröße**

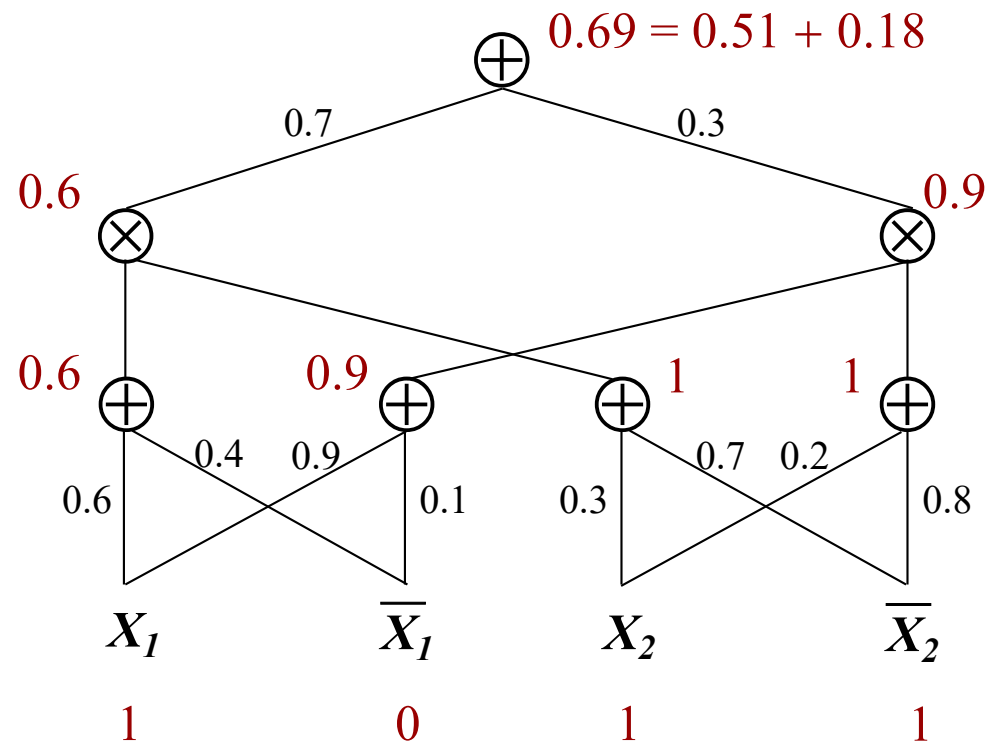


TECHNISCHE
UNIVERSITÄT
DARMSTADT

Marginalwahrscheinlichkeit: $P(q)$

$q: X_1 = 1$

X_1	1
$\overline{X}_1$	0
X_2	1
$\overline{X}_2$	1



TECHNISCHE
UNIVERSITÄT
DARMSTADT

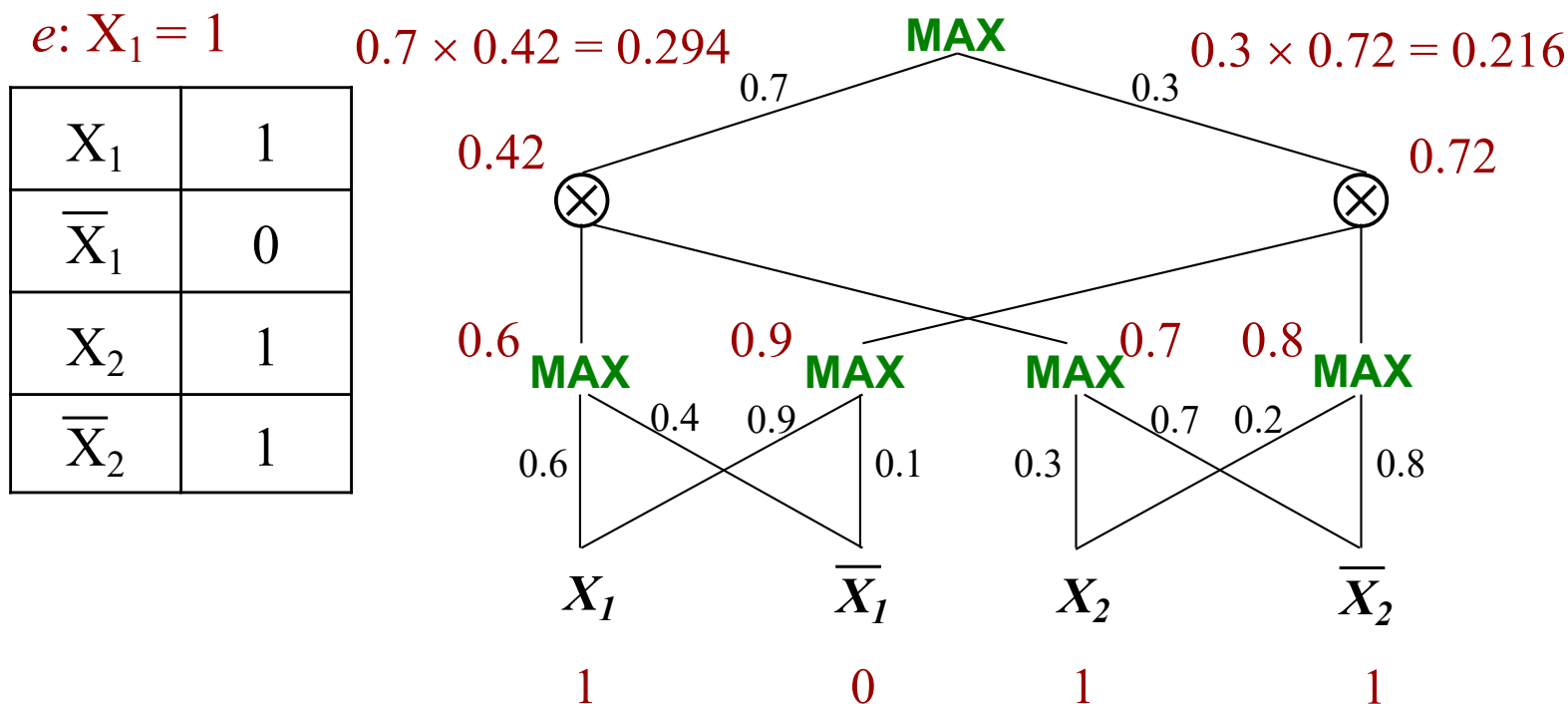


Schlussfolgern: **Linear in der Eingabegröße**



TECHNISCHE
UNIVERSITÄT
DARMSTADT

(Approx.) wahrscheinlichste Vervollständigung:
Ersetzte Summen durch Maximierungen



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Parameterschätzung a la Tiefes Neuronales Lernen

- **Starte mit einem dichten SPN.**
- **Finde die Struktur durch (Online-)Lernen der Gewichte.**
- **Null-Gewichte bedeuten das Fehlen von Verbindungen.** (Hard) EM ist vorteilhaft, um verschwindende Gradienten (Gradient Vanishing) zu vermeiden.
- **Jeder Summenknoten ist eine Mischung über Kindknoten.** Im Prinzip können Sie ein gegebenes SPN in einen TensorFlow-Berechnungsgraphen umwandeln und von dort aus jeden bekannten Algorithmus anwenden.

Modellselektion / Strukturlernen

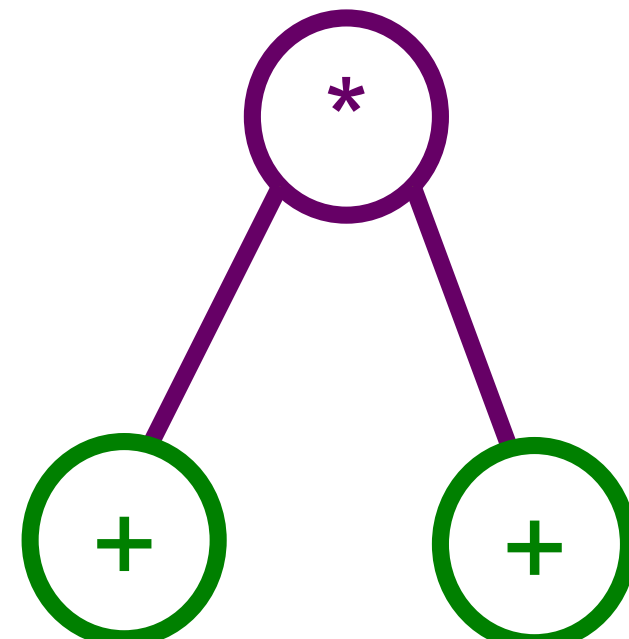
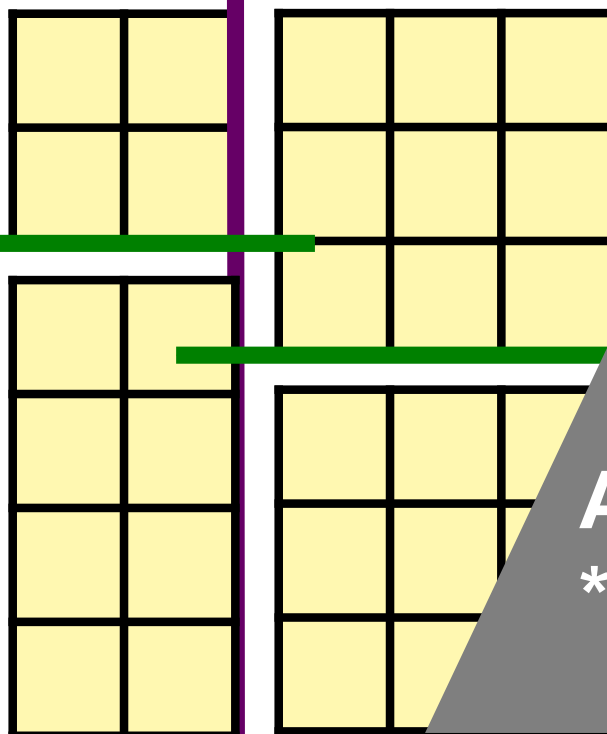
Summen-Produkt Netzwerke

Teste die Unabhängigkeit von
Zufallsvariables

Zufallsvariablen

Beispiel

“Bedingung”
durch Clustering
oder zufällige
Splitte



Abwechselnd
* und + Schichten

[Poon, Domingos UAI'11; Molina, Natarajan, Kersting AAAI'17; Vergari, Peharz, Di Mauro, Molina, Kersting, Esposito AAAI '18; Molina, Vergari, Di Mauro, Esposito, Natarajan, Kersting AAAI '18]

FL $\oplus$ W SPFlow: An Easy and Extensible Library for Sum-Product Networks

[Molina, Vergari, Stelzner, Peharz, Subramani, Poupart, Di Mauro, Kersting 2019]



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



UNIVERSITY OF
WATERLOO



UNIVERSITY OF
CAMBRIDGE



VECTOR
INSTITUTE

CAML

MADESI

DFG



Federal Ministry
of Education
and Research

195 commits

2 branches

0 releases

6 contributors

Branch: master

New pull request

Create new file

Upload files

Find file

Clone or download

<https://github.com/SPFlow/SPFlow>

```
from spn.structure.leaves.parametric.Parametric import Categorical
from spn.structure.Base import Sum, Product
from spn.structure.base import assign_ids, rebuild_scopes_bottom_up

p0 = Product(children=[Categorical(p=[0.3, 0.7], scope=1), Categorical(p=[0.4, 0.6], scope=2)])
p1 = Product(children=[Categorical(p=[0.5, 0.5], scope=1), Categorical(p=[0.6, 0.4], scope=2)])
s1 = Sum(weights=[0.3, 0.7], children=[p0, p1])
p2 = Product(children=[Categorical(p=[0.2, 0.8], scope=0), s1])
p3 = Product(children=[Categorical(p=[0.2, 0.8], scope=0), Categorical(p=[0.3, 0.7], scope=1)])
p4 = Product(children=[p3, Categorical(p=[0.4, 0.6], scope=2)])
spn = Sum(weights=[0.4, 0.6], children=[p2, p4])

assign_ids(spn)
rebuild_scopes_bottom_up(spn)

return spn
```

**Domain Specific Language,
Inference, EM, und
Modellselektion als auch
Kompilation von SPNs nach TF
und PyTorch und auch in anderen
Code: C/C++, GPU, FPGA**

SPFlow, an open-source Python library providing a simple interface to inference, learning and manipulation routines for deep and tractable probabilistic models called Sum-Product Networks (SPNs). The library allows one to quickly create SPNs both from data and through a domain specific language (DSL). It efficiently implements several probabilistic inference routines like computing marginals, conditionals and (approximate) most probable explanations (MPEs) along with compiling

Zufällige SPNs



TECHNISCHE
UNIVERSITÄT
DARMSTADT



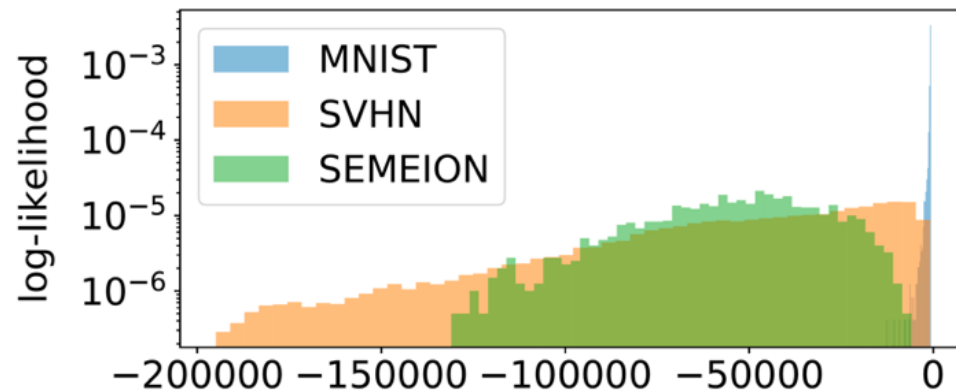
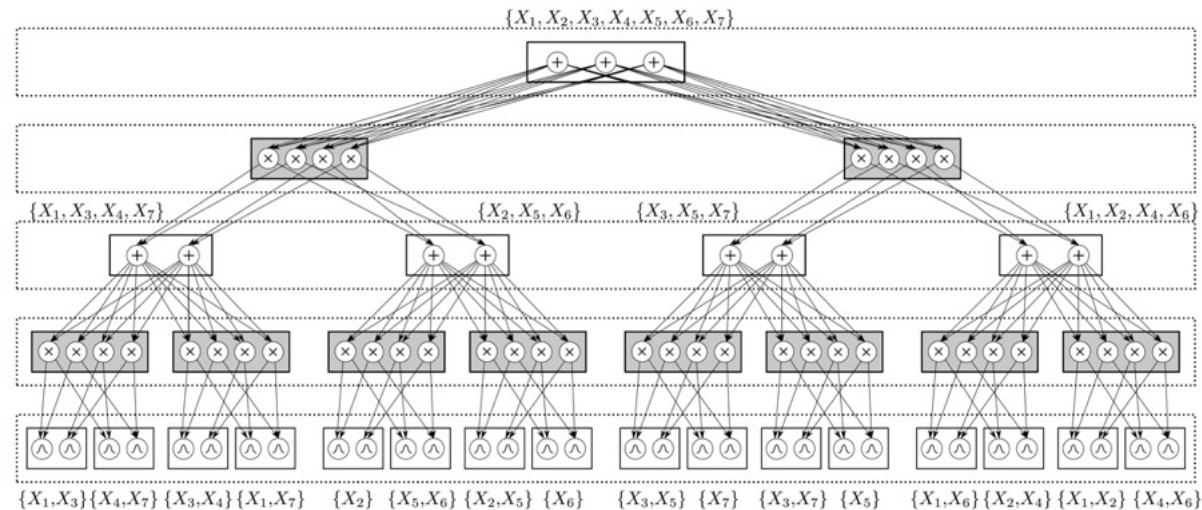
UBER AI Labs



UNIVERSITY OF
CAMBRIDGE



Max Planck Institute for
Intelligent Systems

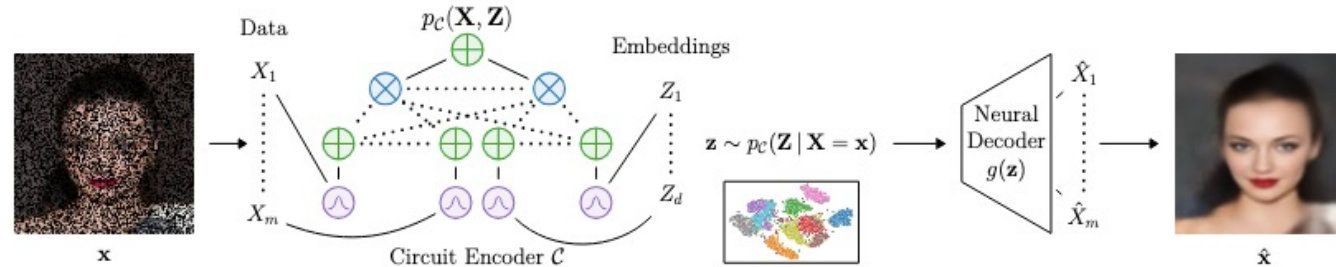


TECHNISCHE
UNIVERSITÄT
DARMSTADT



Autoencoding Probabilistic Circuits (APCs)

[Braun, Sidheekh, Vergari, Mundt, Natarajan, Kersting TMLR 2025]



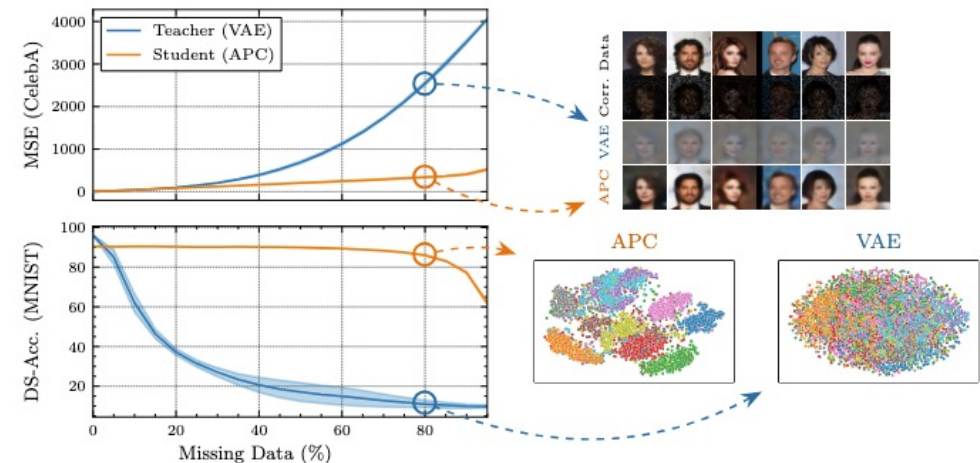
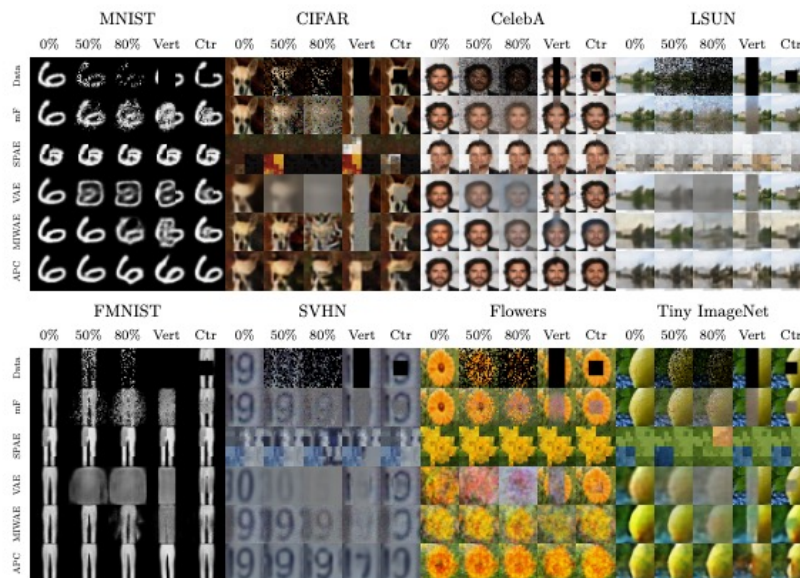
TECHNISCHE
UNIVERSITÄT
DARMSTADT



THE UNIVERSITY
OF TEXAS AT DALLAS



Universität
Bremen



Während neuronale Encoder-Modelle bei einer Zunahme des Anteils fehlender Daten schnell kollabieren und meist an zufällig auftretenden Datenverlusten scheitern, sind APCs in der Lage, eine stabile Rekonstruktion aufrechterhalten.

APCs sind robuste, datenfreie Wissensdestillatoren.

Was haben Sie bisher kennengelernt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

- Die meisten Tiefen Neuronalen Netzwerken stellen keine Verbundwahrscheinlichkeit dar
- Summen-Produkt-Netzwerke sind (tiefe) Berechnungsgraphen für Verbundwahrscheinlichkeiten
- Sie können einem sagen, wenn Sie etwas nicht wirklich wissen



TECHNISCHE
UNIVERSITÄT
DARMSTADT

